

Chapter 8

Application of NLP

Overview:

1. Sentiment Analysis
2. Summary Generation
3. Question Answering
4. Machine Translation

8.1 Sentiment Analysis

8.1 Sentiment Analysis

- **Sentiment Analysis** is a use case of **Natural Language Processing (NLP)** and comes under the category of **text classification**.
- To put it simply, Sentiment Analysis **involves classifying a text into various sentiments**, such as **positive** or **negative**, **Happy**, **Sad** or **Neutral**, etc.
- Thus, the ultimate goal of sentiment analysis is to decipher the underlying mood, emotion, or sentiment of a text.
- This is also known as **Opinion Mining**.

Example 1: Sentiment Analysis

- One common application of NLP is sentiment analysis, which involves analyzing a piece of text to determine the sentiment or emotion expressed within it.
- Let's consider an example to illustrate how sentiment analysis works:
- Text: "I absolutely loved the movie! The plot was captivating, and the acting was superb."
- Sentiment: Positive
- In this example, sentiment analysis would classify the sentiment of the text as positive because the language used ("loved," "captivating," "superb") indicates a positive opinion or emotion towards the movie.

Example 2: Sentiment Analysis..

- Text: "The customer service was terrible. I waited on hold for over an hour, and the representative was rude and unhelpful."
- Sentiment: Negative
- In this case, sentiment analysis would classify the sentiment of the text as negative due to the negative language used ("terrible," "rude," "unhelpful") expressing dissatisfaction with the customer service experience.

Example 3: Sentiment Analysis..

- Text: "The weather forecast for tomorrow is partly cloudy with a chance of rain."
- Sentiment: Neutral
- Here, sentiment analysis would classify the sentiment of the text as neutral because it describes a factual statement about the weather without expressing any particular emotion or opinion.

Sentiment Analysis



Fragrance-1
(Lavender)

- REVIEWS**
1. Smells amazing! A perfect purchase :)
 2. Must buy! Super amazing.
 3. Quite satisfactory



Fragrance-1
(Rose)

- REVIEWS**
1. A decent purchase
 2. Quite okayish! Smells average
 3. Could have been better in lot terms



Fragrance-1
(Lemon)

- REVIEWS**
1. An absolute waste of money.
 2. Total waste of money
 3. Terrible smell, not worth buytng

SENTIMENT ANALYZER



POSITIVE (81%)



NEUTRAL (88%)



negative (91%))

How Does Sentiment Analysis Work?

- Sentiment analysis typically works by employing natural language processing (NLP) techniques to analyze and understand the sentiment expressed in text.
- The process involves several steps:
 1. Text Preprocessing
 2. Tokenization
 3. Feature Extraction
 4. Sentiment Classification
 5. Post-processing
 6. Evaluation

1. **Text Preprocessing:** The text data is **cleaned by removing irrelevant information**, such as special characters, punctuation, and stopwords.
2. **Tokenization:** The text is **divided into individual words** or tokens to facilitate analysis.
3. **Feature Extraction:** Relevant features are extracted from the text, such as **words, n-grams, or even parts of speech**.
4. **Sentiment Classification:** **Machine learning algorithms** or pre-trained models **are used** to classify the sentiment of each text instance. This can be achieved through supervised learning, where models are trained on labeled data, or through **pre-trained models** that have learned sentiment patterns from large datasets.
5. **Post-processing:** The sentiment analysis results may undergo additional processing, such as aggregating sentiment scores or **applying threshold rules** to classify sentiments as positive, negative, or neutral.
6. **Evaluation:** The performance of the sentiment analysis model is assessed using evaluation metrics, such as **accuracy, precision, recall, or F1 score**.

Types of Sentiment Analysis

- Various types of sentiment analysis can be performed, depending on the specific focus and objective of the analysis.
- Some common types include:
 1. Document-Level Sentiment Analysis
 2. Sentence-Level Sentiment Analysis
 3. Aspect-Based Sentiment Analysis
 4. Entity-Level Sentiment Analysis
 5. Comparative Sentiment Analysis

1. Document-Level Sentiment Analysis:

- This type of analysis **determines the overall sentiment expressed in a document**, such as a review or an article.
- It aims to classify the entire text as positive, negative, or neutral.

2. Sentence-Level Sentiment Analysis:

- Here, the sentiment of **each sentence within a document is analyzed**.
- This type provides a more granular understanding of the sentiment expressed in different text parts.

3. Aspect-Based Sentiment Analysis:

- This approach **focuses on identifying and extracting the sentiment associated with specific aspects or entities** mentioned in the text.
- For example, in a product review, the sentiment towards different features of the product (e.g., **performance, design, usability**) can be analyzed separately.

4. Entity-Level Sentiment Analysis:

- This type of analysis identifies the **sentiment expressed towards specific entities** or targets mentioned in the text, such as people, companies, or products.
- It helps understand the sentiment associated with different entities within the same document.

5. Comparative Sentiment Analysis:

- This approach **involves comparing the sentiment between different entities or aspects** mentioned in the text.
- It aims to identify the relative sentiment or preferences expressed towards various entities or features.

Real world application of Sentiment Analysis

- **Assume you start a company that sells perfumes** on an online platform. You put up a wide range of fragrances out there and soon customers start flooding in.
- After some time **you decide to change the pricing strategy** of perfumes — you plan to increase the prices of the popular fragrances and at the same time offer discounts on unpopular ones.
- Now, in order to determine **which fragrances are popular**, you start going through **customer reviews** of all the fragrances.
- But you're stuck! They are just so many that you cannot go through them all in one lifetime.
- **This is where sentiment analysis** can rope you out of the pit.
- You simply gather all the reviews in one place and apply sentiment analysis to it.

Sentiment Analysis: Case Example



- From these results, we can clearly see that:
 - **Fragrance-1 (Lavender)** has highly **positive** reviews by the customers which indicates your company can **escalate its prices** given its popularity.
 - **Fragrance-2 (Rose)** happens to have a **neutral** outlook amongst the customer which means your company **should not change its pricing**.
 - **Fragrance-3 (Lemon)** has an overall **negative** sentiment associated with it — thus, your company should **consider offering a discount** on it to balance the scales.
 - This was just a simple example of how sentiment analysis can help you gain insights into your products/services and help your organization make decisions.

Sentiment Analysis Use Cases

1. Social Media Monitoring for Brand Management:

- Brands can use sentiment analysis to gauge their Brand's public outlook.
- For example, a company can gather all Tweets with the company's mention or tag and perform sentiment analysis to learn the company's public outlook.

2. Product/Service Analysis:

- Brands/Organizations can perform sentiment analysis on customer reviews to see how well a product or service is doing in the market and make future decisions accordingly.

3. Stock Price Prediction:

- Predicting whether the stocks of a company will go up or down is crucial for investors.
- One can determine the same by performing sentiment analysis on News Headlines of articles containing the company's name.
- If the news headlines pertaining to a particular organization happen to have a positive sentiment — its stock prices should go up and vice-versa.

Python tools for Sentiment Analysis

- Python is one of the most powerful tools when it comes to performing data science tasks.
- It offers various ways to perform sentiment analysis.
- The most popular ones are enlisted here:
 1. Using Text Blob
 2. Using Vader
 3. Using Bag of Words Vectorization-based Models
 4. Using LSTM-based Models
 5. Using Transformer-based Models

i) Text Blob

- **Text Blob** is a Python library for Natural Language Processing.
- Using Text Blob for sentiment analysis is quite simple.
- It **takes text as an input** and can **return polarity** and **subjectivity** as outputs.
 - **Polarity :**
 - determines the sentiment of the text.
 - Its values lie in $[-1,1]$
 - where -1 denotes a highly negative sentiment and
 - 1 denotes a highly positive sentiment.
 - **Subjectivity:**
 - determines whether a text input is factual information or a personal opinion.
 - Its value lies between $[0,1]$
 - where a value closer to 0 denotes a piece of factual information and
 - a value closer to 1 denotes a personal opinion.

```
1. from textblob import TextBlob
2. text_1="The movie was so awesome."
3. text_2= "The food here tastes terrible."

4. # Determining the polarity
5. p_1=TextBlob(text_1).sentiment.polarity
6. p_2=TextBlob(text_2).sentiment.polarity

7. # Determining the subjectivity
8. s_1=TextBlob(text_1).sentiment.subjectivity
9. s_2=TextBlob(text_2).sentiment.subjectivity
10.
11. print("Polarity of text_1 is",p_1)
12. print("Polarity of text_2 is",p_2)
13. print("Subjectivity of text_1 is",s_1)
14. print("Subjectivity of text_2 is",s_2)
```

Output:

```
Polarity of text_1 is 1.0
Polarity of text_2 is -1.0
Subjectivity of text_1 is 1.0
Subjectivity of text_2 is 1.0
```

```
from textblob import TextBlob
from google.colab import drive
drive.mount('/content/gdrive')
#Reading data
f = open('/content/gdrive/MyDrive/1. Teaching Materials/GCES/NLP/FeedBackData.txt')
for line in f:
    print(line)
    obj=TextBlob(line)
    sentiment=obj.sentiment.polarity

    if(sentiment<0):
        print("the sentence is negative\n")
    elif(sentiment==0):
        print('This sentence is neutral\n')
    elif(sentiment>0):
        print('This sentence is positive\n')
```

The flower of rose is beautiful.

This sentence is positive

This product was terrible.

the sentence is negative

I was going to that place.

This sentence is neutral

He is so annoying.

the sentence is negative

This product is nice and have many features.

This sentence is positive

ii) VADER

- **VADER (Valence Aware Dictionary and sEntiment Reasoner)** is a rule-based sentiment analyzer that has been trained on social media text.
- Just like Text Blob, its usage in Python is pretty simple.

```
pip install vaderSentiment
```

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
sentiment = SentimentIntensityAnalyzer()
text_1 = "The book was a perfect balance between wrtiting style and plot."
text_2 = "The pizza tastes terrible."
sent_1 = sentiment.polarity_scores(text_1)
sent_2 = sentiment.polarity_scores(text_2)
print("Sentiment of text 1:", sent_1)
print("Sentiment of text 2:", sent_2)
```

Sentiment of text 1: {'neg': 0.0, 'neu': 0.73, 'pos': 0.27, 'compound': 0.5719}

Sentiment of text 2: {'neg': 0.508, 'neu': 0.492, 'pos': 0.0, 'compound': -0.4767}

General steps for building a sentiment analyzer

1. **Load the dataset:** The CSV file is loaded into a pandas DataFrame.
2. **Preprocess the text data:** The text is converted to lowercase, punctuation is removed, and stopwords are removed.
3. **Vectorize the text data:** The text data is converted into a bag-of-words model using CountVectorizer, TFIDFVectorizer, and so on.
4. **Split the dataset:** The data is split into training and testing sets.
5. **Train the classifier:** A Naive Bayes classifier is trained on the training set.
6. **Make predictions:** The classifier makes predictions on the test set.
7. **Evaluate the classifier:** The accuracy and classification report (precision, recall, F1-score) are printed.
8. **Test with unseen new data.**

8.2 Summary Generation

Text Summarization

- When you open news sites, **do you just start reading every news** article? **Probably not.**
- We typically glance the short news summary and then read more details if interested.
- Short, informative summaries of the news is now everywhere like magazines, news aggregator apps, research sites, etc.
- Well, It is possible to create the summaries automatically as the news comes in from various sources around the world.
- The **method of extracting these summaries from the original huge text without losing vital information is called as Text Summarization.**
- Text summarization in NLP involves creating a concise and coherent version of a longer text while preserving its essential information and meaning.

Types of Text Summarization

- Text summarization methods can be grouped into two main categories:
 - **Extractive** and
 - **Abstractive methods**

1. Extractive Summarization

Extractive summarization works by selecting and concatenating important sentences or phrases directly from the source text.

Techniques:

- **Frequency-based Methods:** Select sentences with the most frequent terms.
- **Graph-based Methods:** Use algorithms like TextRank to rank sentences based on their connectivity in a graph representation of the text.
- **Machine Learning-based Methods:** Train models to identify key sentences using features like term frequency, sentence position, etc.

Example Algorithm: TextRank

- **Preprocessing:** Tokenize the text, remove stop words, and normalize the text.
- **Graph Construction:** Represent sentences as nodes and edges based on sentence similarity (e.g., cosine similarity of TF-IDF vectors).
- **Ranking:** Apply the PageRank algorithm to rank sentences based on their importance.
- **Selection:** Choose the top-ranked sentences for the summary.

2. Abstractive Summarization

Abstractive summarization generates new sentences that capture the essence of the source text, potentially rephrasing and restructuring information.

Techniques:

- **Sequence-to-Sequence Models (Seq2Seq):** Use models with encoder-decoder architecture, often with attention mechanisms.
- **Transformers:** Utilize transformer-based models like BERT, GPT, or BART for generating summaries.

i) Extractive Text Summarization

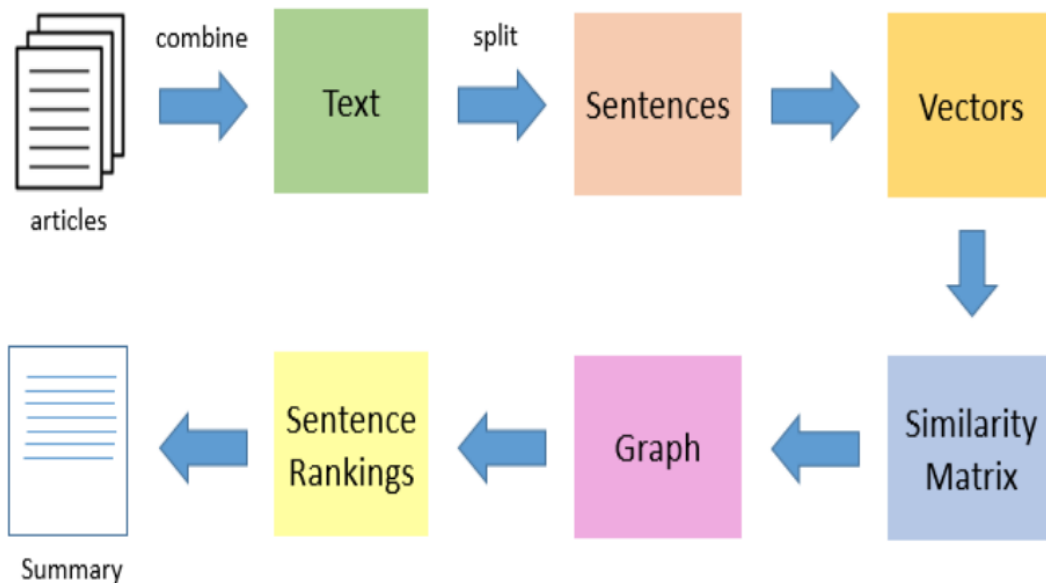
- It is the traditional method developed first.
- The main objective is to **identify the significant sentences of the text and add them to the summary.**
- The summary obtained **contains exact sentences from the original text.**
-

ii) Abstractive Text Summarization

- It is a more advanced method.
- The approach is to **identify the important sections, interpret the context and reproduce in a new way.**
- This ensures that the **core information is conveyed through shortest text possible.**
- Note that here, the sentences in summary are generated, not just extracted from original text.
- Abstractive summarization is the new state of art method, which generates new sentences that could best represent the whole text.
- This is better than extractive methods where sentences are just selected from original text for the summary.

Workflow for Text Summarization

- **Data Collection:** Gather a dataset of documents and corresponding summaries.
- **Preprocessing:** Clean and preprocess the text (tokenization, stop word removal, etc.).
- **Model Selection:** Choose an extractive or abstractive summarization model based on the use case.
- **Training:** Train the model on the dataset, fine-tuning pre-trained models if applicable.
- **Inference:** Use the trained model to generate summaries for new documents.
- **Evaluation:** Assess the quality of the summaries using metrics like ROUGE.



8.3 Question Answering

8.3 Question Answering System

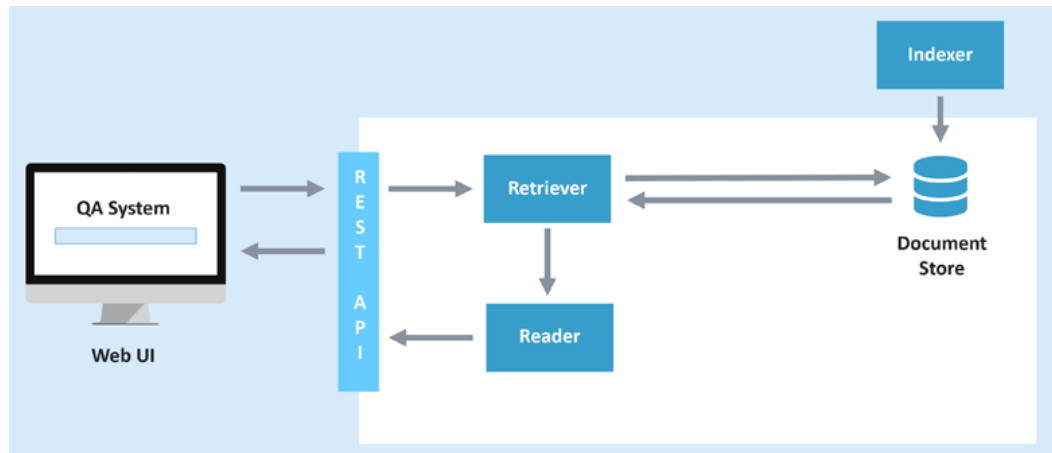
- Unlike traditional keyword searches, a Natural Language Question Answering system does not return a complete document to the user. Instead, users **ask a question in natural language** and receive a specific answer in return.



Figure 1: A natural language question answering system takes questions in natural language and returns a concrete answer

Question Answering System

1. The document store is responsible for providing relevant documents. There are different ways of doing this.
2. A retriever is responsible for retrieving relevant documents for the user's question. First of all, it attempts to **extract the relevant terms in the question**. It then uses these to retrieve relevant documents.



Open Datasets for Question Answering

1. *Stanford Question Answering Dataset (SQuAD)*
2. **WikiQA** dataset
3. **TREC-QA** dataset
4. **NewsQA** dataset

Open Datasets for Question Answering

1. *Stanford Question Answering Dataset (SQuAD)*

- is a reading comprehension dataset, consisting of questions posed by crowd workers on a set of Wikipedia articles,
- where the answer to every question is a segment of text, or *span*, from the corresponding reading passage, or the question might be unanswerable.

2. **WikiQA** dataset,

- is a publicly available set of question and answer pairs, collected and annotated for research on open-domain question answering.
- It is constructed using a more natural process and is more than an order of magnitude larger than the previous dataset.

3. The **TREC-QA**

- dataset contains questions and answer patterns, as well as a pool of documents returned by participating teams.

4. **NewsQA** dataset

- is to help the research community build algorithms that are capable of answering questions requiring human-level comprehension and reasoning skills.

Types of Question Answering

1. IR-based Factoid **Question Answering** goal is to answer a user's question by finding short text segments on the Web or some other collection of documents.
2. Knowledge-based **question answering** is the idea of answering a natural language question by mapping it to a query over a structured database.
3. **Using multiple information sources: IBM's Watson** system from IBM that relies on a wide variety of resources to answer questions.

Challenges in Question Answering

1. Lexical Gap :

- In a natural language, the **same meaning can be expressed in different ways.**
- Because a question can usually only be answered if every referred concept is identified, bridging this gap significantly increases the proportion of questions that can be answered by a system.

2. Ambiguity :

- It is the phenomenon of the **same phrase having different meanings**; this can be structural and syntactic (like “flying planes”) or lexical and semantic (like “bank”).
- The same string accidentally refers to different concepts (as in money bank vs. river bank) and polysemy, where the same string refers to different but related concepts (as in bank as a company vs. bank as a building).

3. Multilingualism :

- Knowledge on the Web is **expressed in various languages.**
- While RDF resources can be described in multiple languages at once using language tags, there is not a single language that is always used in Web documents.
- Additionally, users have different native languages.
- A Question-Answering system is expected to recognize a language and get the results on the go!

Trending QA system

1. Chat GPT:

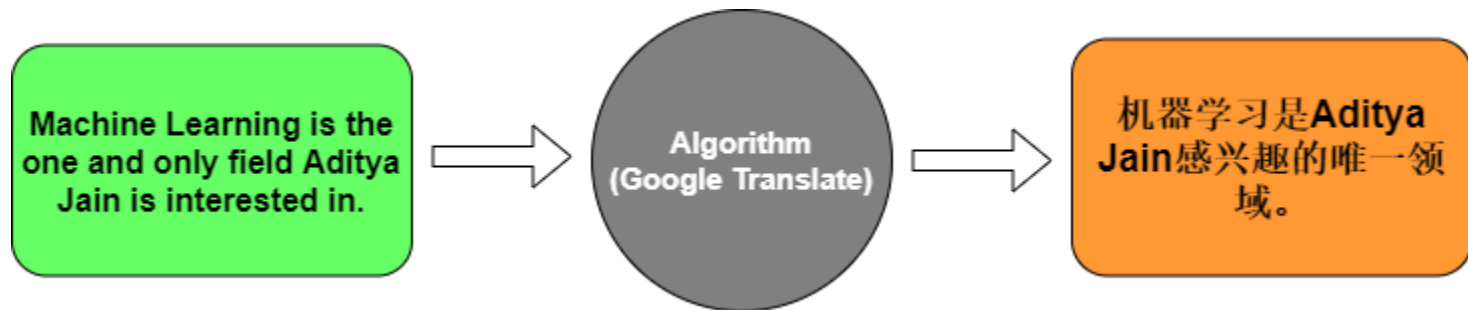
- ChatGPT stands for 'Chat Generative Pre-training Transformer.'
- It's an AI-powered model developed by OpenAI, a research organization aiming to ensure artificial general intelligence (AGI) benefits all of humanity.

2. BERT

- Bidirectional Encoder Representations from Transformers,
- model is an open source machine learning framework for natural language processing ([NLP](#)).
- BERT is designed to help computers understand the meaning of ambiguous language in text by using surrounding text to establish context.

8.4 Machine Translation

8.4 Machine Translation



Machine Translation is the process of converting the text in a source language to a required target language.

Machine Translation

- Machine translation, also known as **robotized interpretation**,
- is **the process in which computers or machines independently and quickly translate vast volumes of text from a particular source language to a target language** without any effort put in by human beings.
- In other words, machine translation functions by employing an application that helps translate text from one input language to another.
- There are four different types of machine translation in NLP:
 - statistical machine translation,
 - rule-based machine translation,
 - hybrid machine translation, and
 - neural machine translation.
- The main advantage of machine translation is its delivery of an effective combination of both speed and cost-effectiveness.

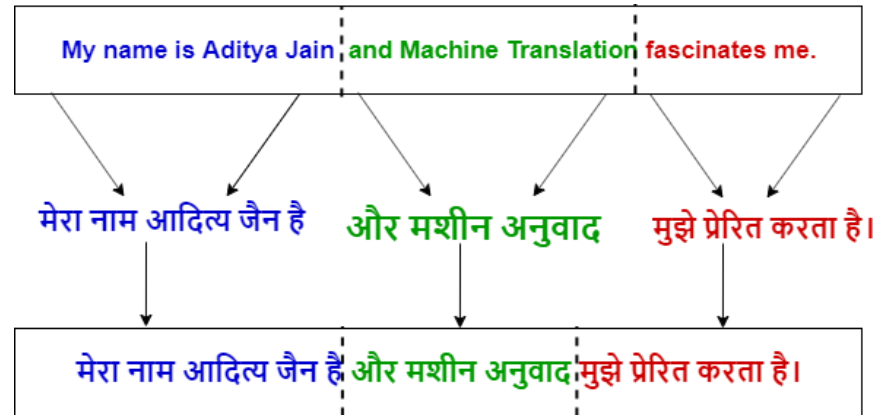
Types of Machine Translation

1. Statistical Machine Translation (SMT):

- Statistical Machine Translation (**SMT**) is a machine translation paradigm where translations are made on the basis of **statistical models**, the parameters of which are derived on the basis of the analysis of large volumes of a **bilingual text corpus**.
- The term **bilingual text corpus** refers to the collection of a large and structured set of texts written in two different languages.
- Supervised and unsupervised machine learning algorithms are used for building statistical models.

2. Rule-Based Machine Translation in NLP:

- Rule-Based Machine Translation (**RBMT**) relies on innumerable **built-in linguistic rules and millions of bilingual dictionaries** for each language pair.



3. Hybrid Machine Translation in NLP (HMT)

- As the name suggests, **HMT** is a method of machine translation that **incorporates the use of multiple different machine translation approaches** within a single machine translation system.
- The underlying motivation behind the use of HMT is the fact that a **failure of a single machine translation technique should not stop the system** from achieving the required level of accuracy.

4. Neural Machine Translation (NMT):

- In Neural Machine Translation (**NMT**), we **make use of a neural network model** to learn a statistical model for machine translation.

End of chapter